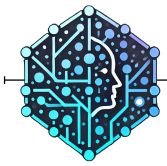


LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. AI APPLICATIONS IN PRACTICE

10 - Docker

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

INTRODUCTION TO DOCKER

The problem

We want to deploy our app from the PC of the developer to production.

We want to replicate the environment, considering all the dependencies (e.g. required libraries, frameworks, supporting tools)

We want to do so easily, cheaply, fastly

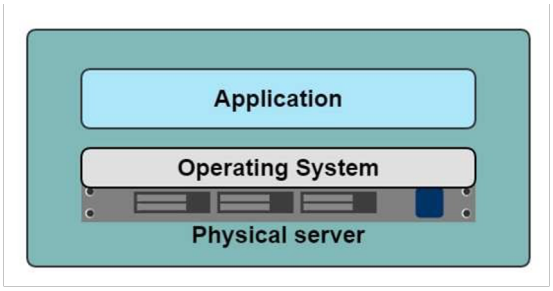
Why Docker Matters for AI Applications

Reproducibility: ensure models run the same way everywhere
Dependency Management: package complex ML library requirements
Scalability: scale machine learning inference services horizontally
Isolation: run multiple model versions without conflicts
Resource Efficiency: better utilization compared to VMs
Deployment Simplicity: streamlined path from development to production

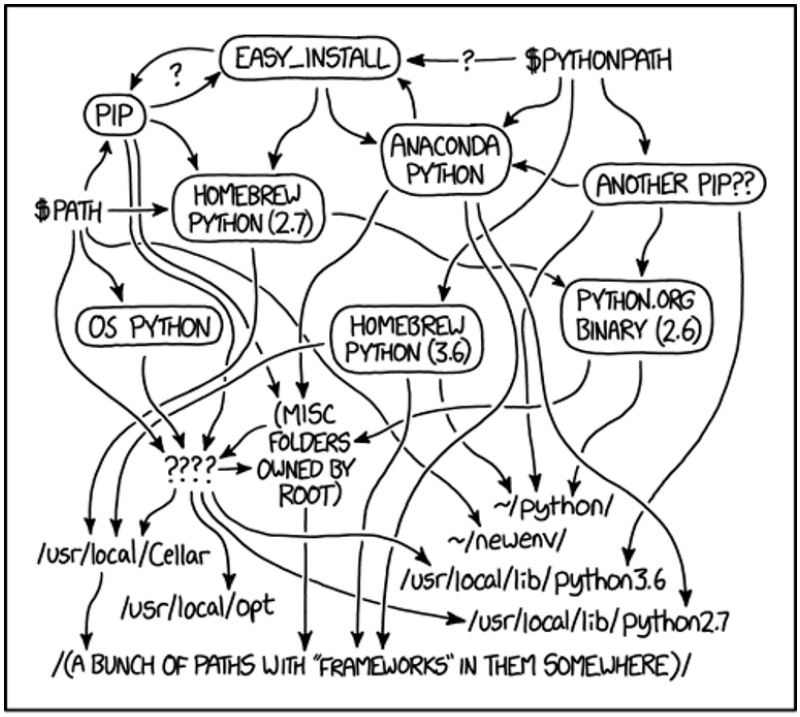
Docker bridges the gap between data scientists and production systems

Deploying applications: straight to physical servers

- Prepare a physical server, install OS; install libraries and app requirements, install app.
- Replicating the whole environment from dev is hard.
- Maintaining envs is hard…



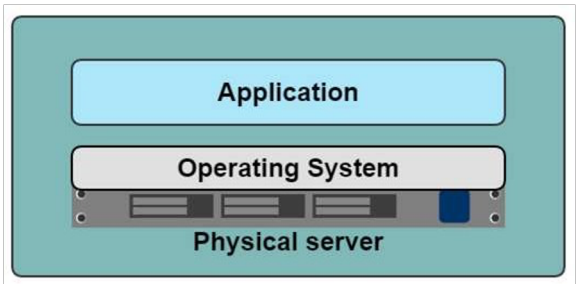
Maintaining Python envs…



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

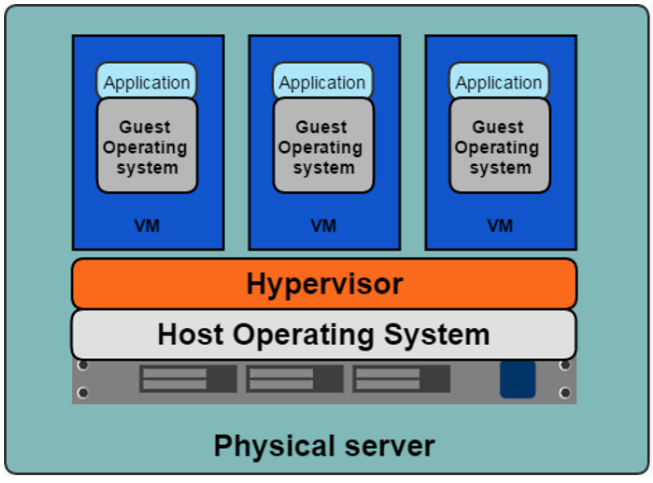
Historical limitations of application deployment

- Slow deployment times
- Huge costs
- Wasted resources
- Difficult to scale
- You have to manage it manually
- Difficult to migrate
- Vendor lock in



Deploying on Hypervisor-based Virtualization

- One physical server can contain multiple applications
- Each application runs in a virtual machine (VM)
- Tools like Vagrant help deploying apps



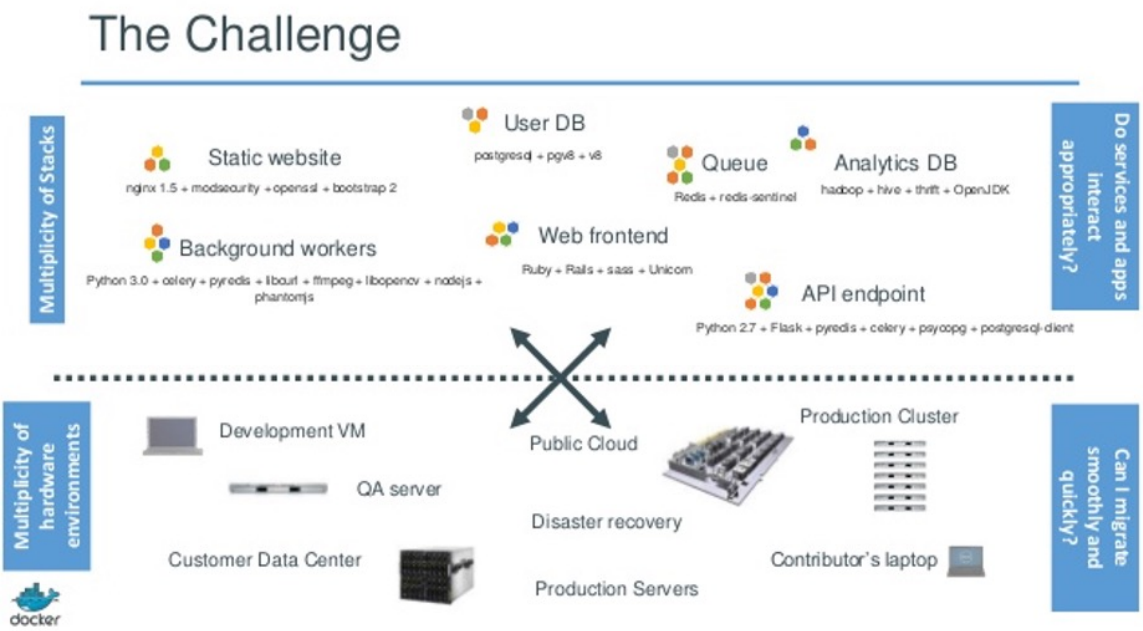
Benefits of VMs

- Better resource pooling
- One physical machine divided into multiple virtual machines
- Easier to scale
- VMs in the cloud
- Rapid elasticity
- Pay as you go model

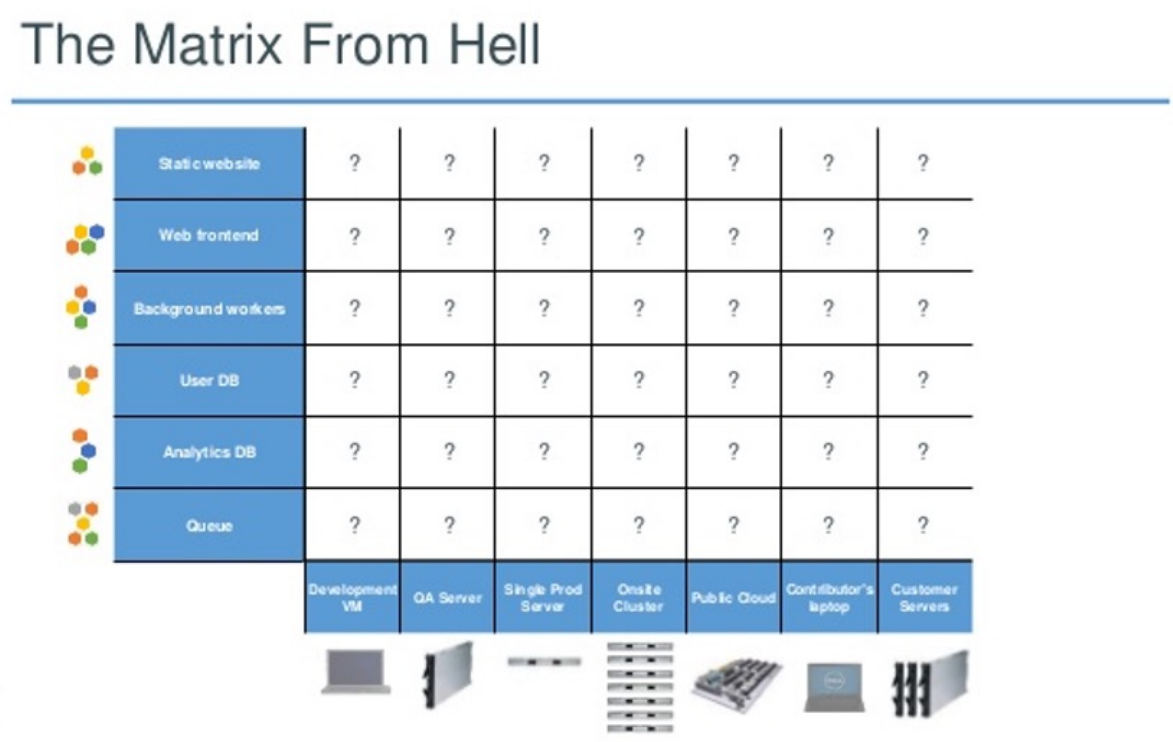
Limitations of VMs

- Each VM stills requires
  - CPU allocation
  - Storage
  - RAM
- An entire guest operating system
- The more VMs you run, the more resources you need
- Guest OS means wasted resources
- Application portability not guaranteed

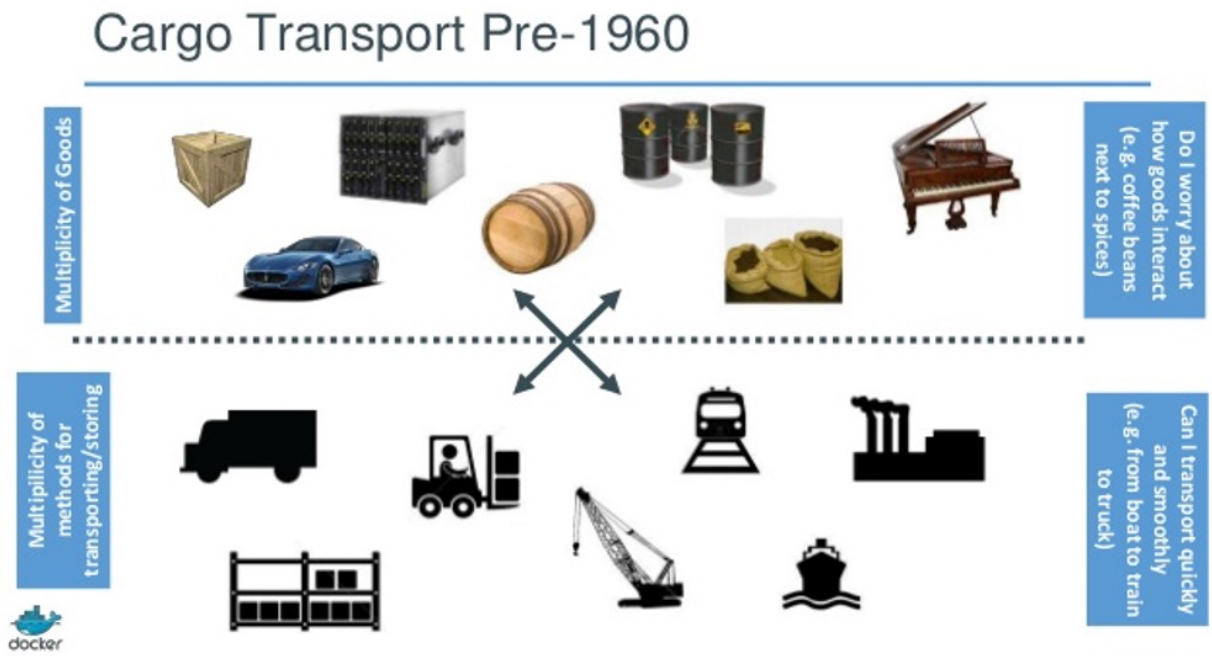
A bit of shipping history…



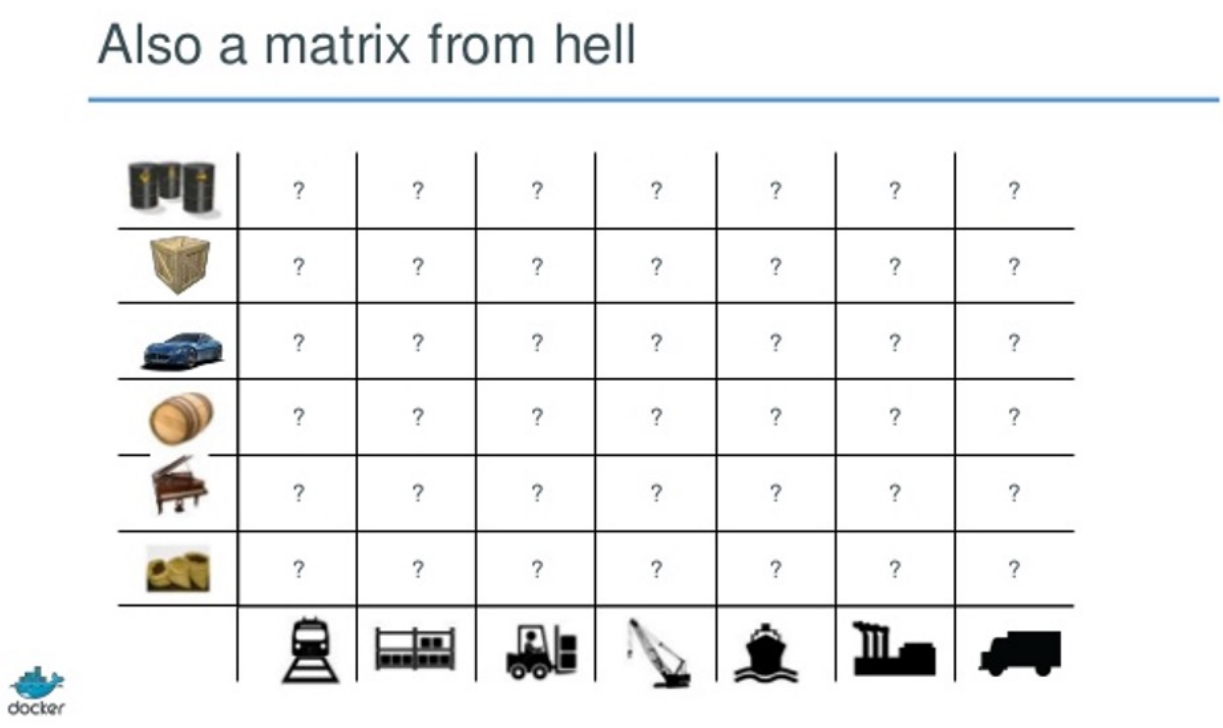
A bit of shipping history…



A bit of shipping history…

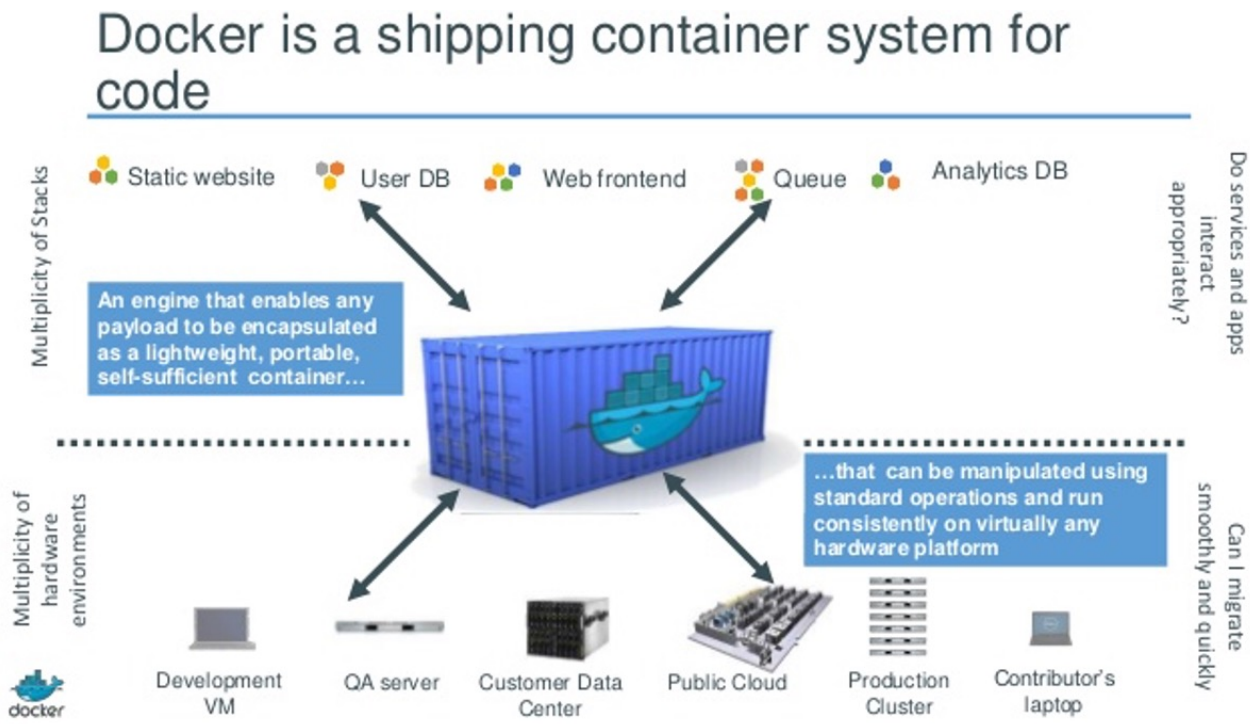


A bit of shipping history…



A practical solution to shipping goods

A practical solution to shipping goods



Introduction to Docker

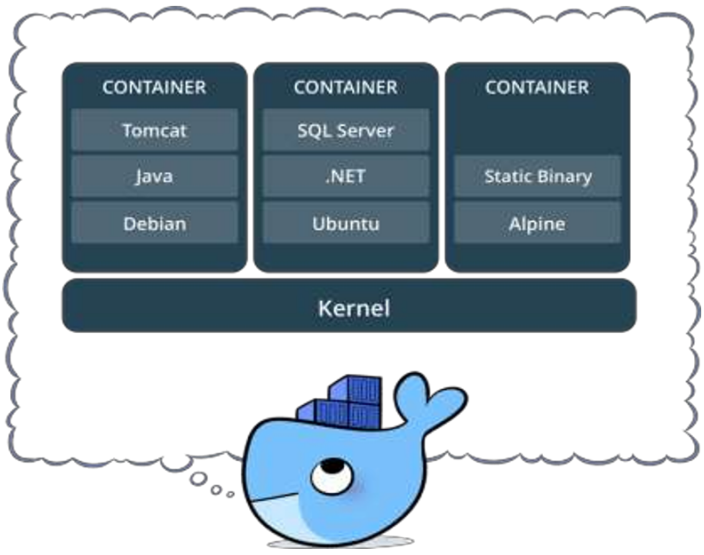
Docker is a platform for developing, shipping, and running applications in isolated environments called containers.

Key features:

- Lightweight and standalone executable packages
- Contains everything needed to run an application
- Consistent environments across development, testing, and production
- Uses host OS kernel but isolates application processes
- More efficient than virtual machines (less overhead)

Container

- Standardized packaging for software and dependencies
- Isolate apps from each other
- Share the same OS kernel
- Works with all major Linux and Windows Server



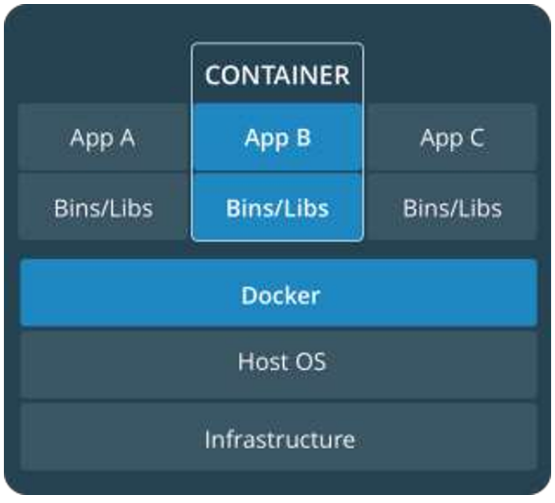
Containers vs Virtual Machines

Containers and virtual machines both provide isolation, but work differently:

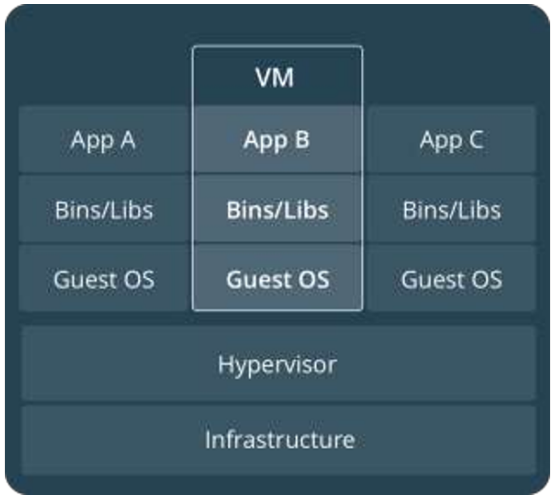
Virtual Machines:
- Run a complete operating system with its own kernel
- Require significant resources (memory, CPU, storage)
- Slower to start (minutes)
- Complete isolation at hardware level

Containers:
- Share the host operating system kernel
- Lightweight (megabytes vs gigabytes)
- Start in seconds
- Process-level isolation

Container vs. VM

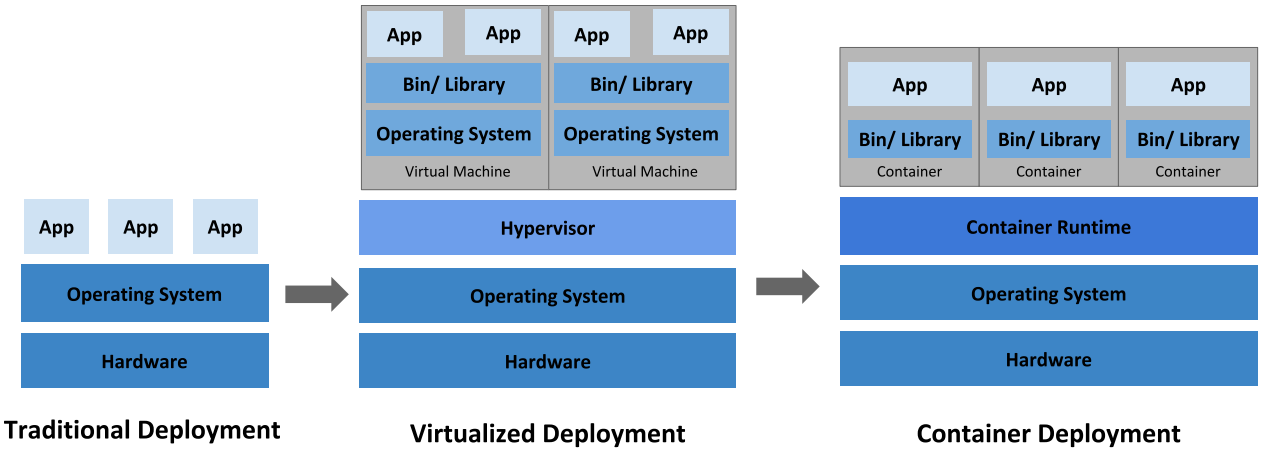


Containers are an app level construct



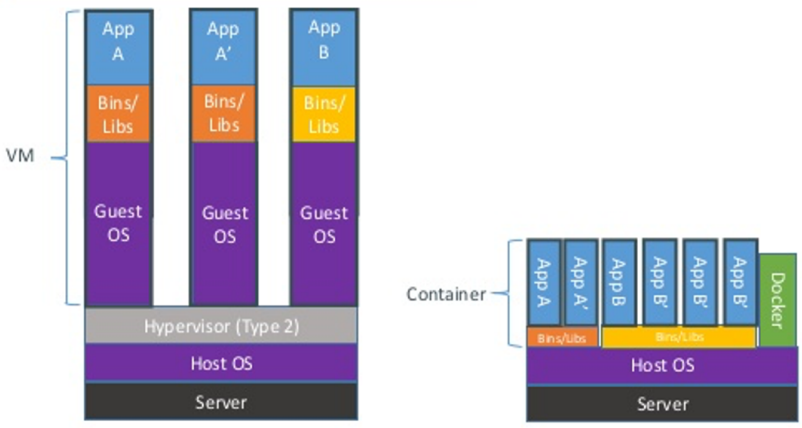
VMs are an infrastructure level construct to turn one machine into many servers

Container vs. VM



Benefits of containers

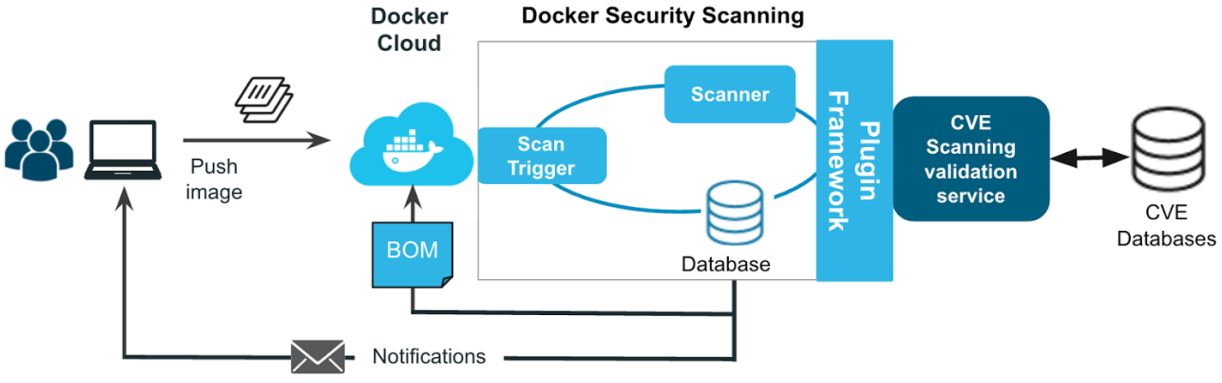
- Containers are isolated but share OS and, where appropriate, bins/libraries:
  - Faster deployment
  - Less overhead
  - Easier migrations
  - Faster start/restart
- Process isolation
  - Separated execution environment
  - File system isolation



Benefits of containers

- Speed
  - No OS to boot = applications online in seconds
- Portability
  - Less dependencies between process layers = ability to move between infrastructure
  - Build your runtime whatever you want. Test it and then move it into the production environment.
- Efficiency
  - Less OS overhead
  - Improved VM density
- Security
  - Images are scanned for CVE vulnerabilities

Security scanning



A bit about Unix OS architecture

A Unix operating system is broken into two primary components, **the kernel space**, and **the user space**.

- The kernel talks to the hardware and provides core system features.
- The user space is the environment that people are most familiar with. It is where applications, libraries and system services run.

Users use an operating system that has a fixed combination of kernel and user space.

- If you have access to a machine running CentOS then you cannot install software that was packaged for Ubuntu on it, because the user space of these distributions is not compatible.
- It can also be very difficult to install multiple versions of the same software, which might be needed to support reproducibility in different workflows over time.

Containers and user space

Containers change the user space into a swappable component.

This means that the entire user space portion of a Linux operating system, including:

- programs, custom configurations, and environment can be independent of whether your
- system is running CentOS, Fedora etc., underneath.

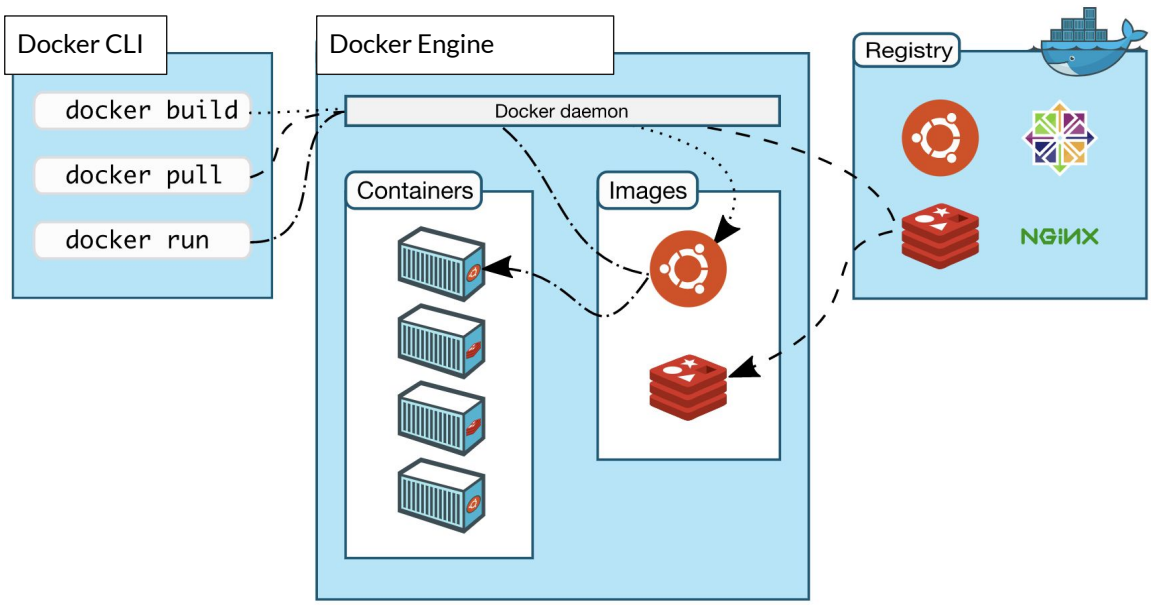
Software developers can build their stack onto whatever operating system base fits their needs best and create distributable runtime environments so that users never have to worry about dependencies and requirements, that they might not be able to satisfy on their systems.

Use cases

- Install a new development environment (stack framework, etc.)
- Replicate a development environment (to a colleague or to another server)
- Share an application and run it quickly regardless of the operating system
- Resume the same project after some time, even if new libraries have been released
- Work simultaneously on multiple projects using the same software (e.g. Python) but with different versions and/or libraries
- Quickly bring software into staging and production in dev

DOCKER ELEMENTS

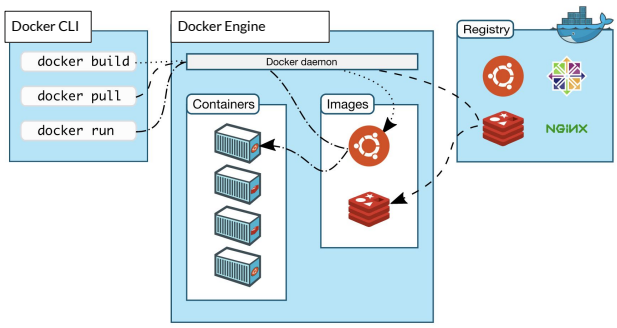
Docker architecture



Docker architecture

Docker uses a client-server architecture:

- **Docker Client**: command-line interface for interacting with Docker
- **Docker Host**: runs the Docker daemon (dockerd)
- **Docker Daemon**: manages Docker objects (images, containers, networks, volumes)
- **Docker Registry**: stores Docker images (Docker Hub is the default public registry)



Docker objects

Image

•The basis of a Docker container. Read-only templates with instructions for creating containers

Container

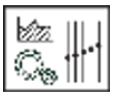
•The image when it is 'running.' Runnable instances of images (isolated environments)

Engine

•The software that executes commands for containers. Networking and volumes are part of Engine. Can be clustered together.

Registry

•Stores, distributes and manages Docker images



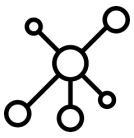
Control Plane

•Management plane for container and cluster orchestration

Docker objects

Volumes

• Persistent data storage outside containers



Networks

• Communication channels between containers

Dockerfile

• Script with instructions to build an image

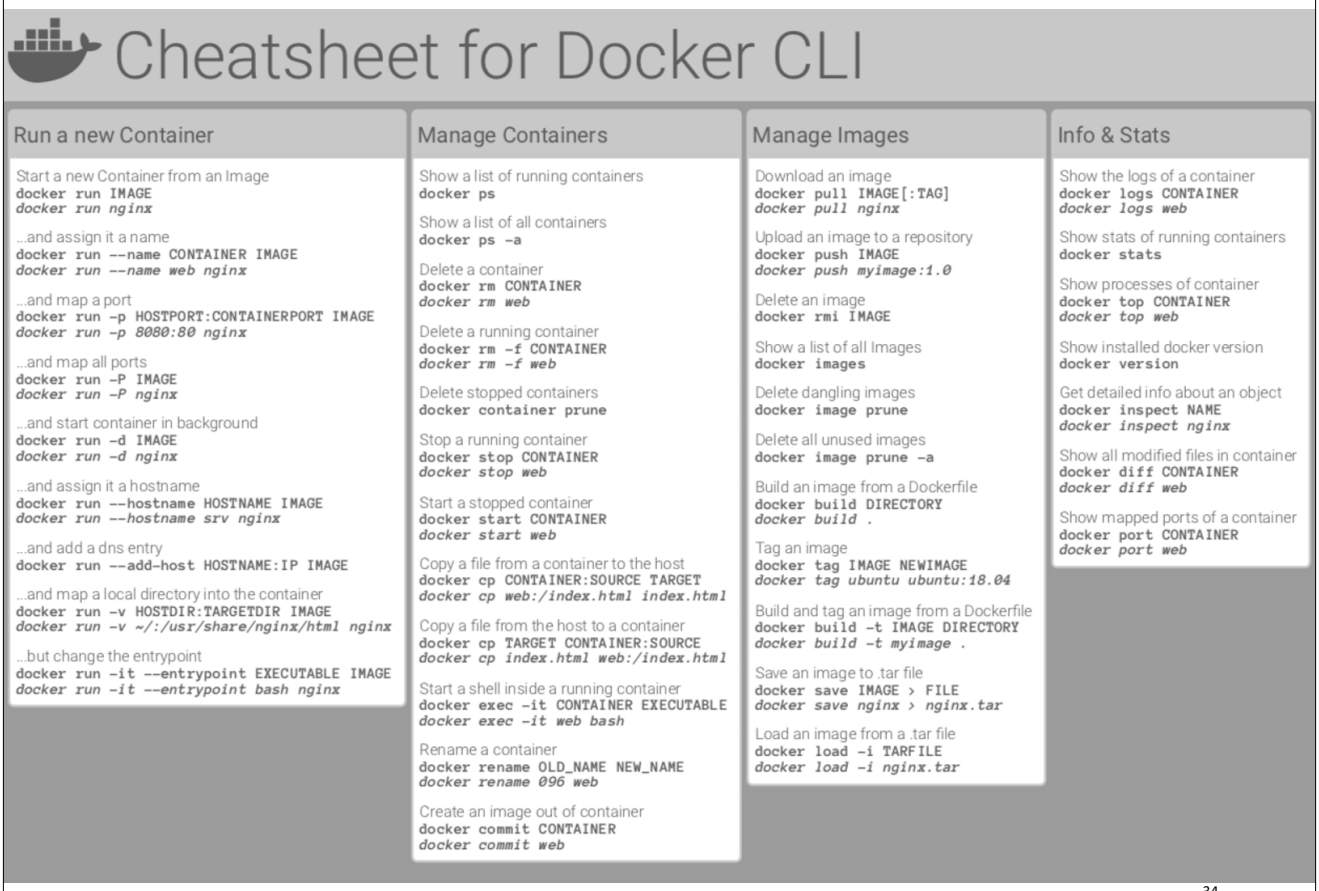
Docker installation

Docker can be installed on various operating systems:

Windows:
- Docker Desktop for Windows (requires WSL2)
- Requires Windows 10 Pro/Enterprise/Education or Windows 11

macOS:
- Docker Desktop for Mac (Intel or Apple Silicon)

Linux:
- Distribution-specific packages (look up on docker website)
- Ubuntu: `sudo apt install docker.io`
- Fedora: `sudo dnf install docker`



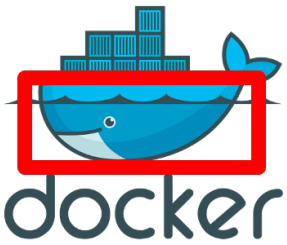
Docker engine

Docker Engine is the heart of the Docker system.

It is a server-side application that takes care of creating, running, and managing Docker containers.

It consists of three main parts:

- the Docker server,
- the Docker API,
- the Docker CLI.



The Docker server is the main process that manages all containers, the Docker API provides an interface to interact with the Docker daemon, and the Docker CLI is the command-line interface we use to give instructions to the Docker daemon.

Docker image

Docker images are **read-only templates** that contain instructions for creating a container.

A Docker image is a standardized package that includes all of the files, binaries, libraries, and configurations to run a **container**.

Images are **immutable**. Once an image is created, it can't be modified. You can only make a new image or add changes on top of it.

Container images are composed of **layers**. Each layer represented a set of file system changes that add, remove, or modify files.

Docker registry

A Docker registry is a storage and distribution system for named Docker images. The same image might have multiple different versions, identified by their tags.

A Docker registry is organized into Docker repositories, where a repository holds all the versions of a specific image.

> The registry allows Docker users to pull images locally, as well as push new images to the registry (given adequate access permissions when applicable).

By default, the Docker engine interacts with **DockerHub**, Docker's public registry instance.

However, it is possible to run on-premise the open-source Docker registry/ distribution.

Exercise

Install Docker on your machine. With the CLI interface try:

- **docker images**

Expected result: show images in your docker environment, with info regarding repository, tags and size

- **docker search alpine**

Expected result: search for images that are on the Docker Hub related to the alpine term.

- **docker pull alpine**

Expected result: an image is downloaded and becomes visible when running docker images

- **docker rmi alpine**

Expected result: the Linux Alpine image is deleted and doesn't show again when running docker images

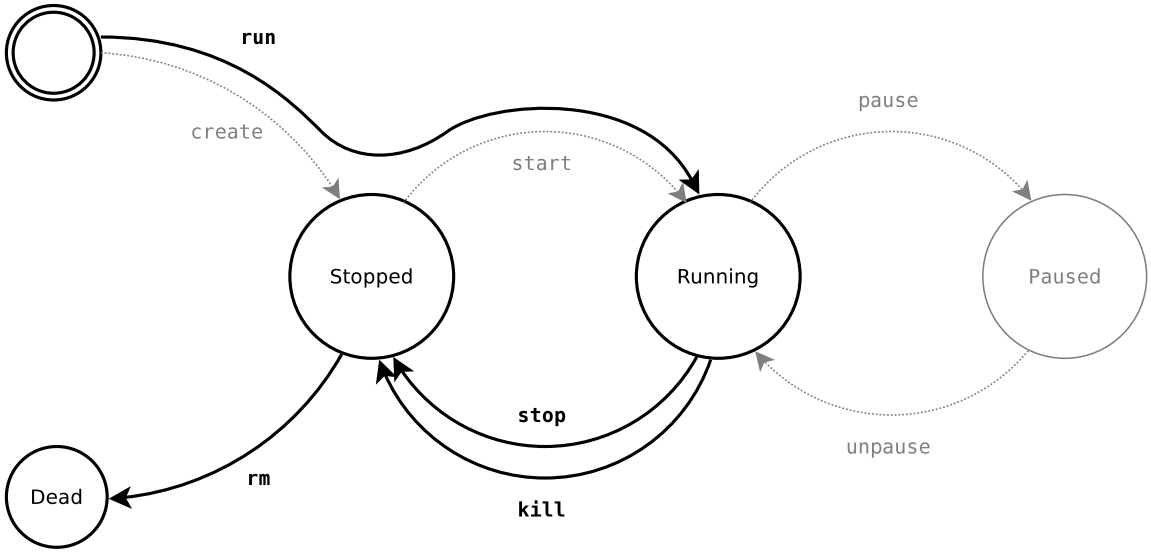
Docker container

Docker **images** become **containers** at runtime – when they run on Docker Engine.

Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging.

- Containers are isolated processes for each of your app's components.
- Docker will assign a name to running containers so we can check their status or run commands on specific instances of images.

Container life cycle



The image/container filesystem

- A docker image is an immutable snapshot of the filesystem
- A docker container is a temporary file system
- layered over an immutable fs (docker image)
- fully writable (copy-on-write)
- dropped at container's end of life (unless a commit is made)
- the root filesystem is created in create and dropped in rm (it is persistent across stop/start)

Docker run

There are many different ways to run a container. By adding attributes to the basic syntax, you can configure a container to run in detached mode, set a container name, mount a volume, and perform many more tasks.

docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

To check all these parameters:

docker run --help

Exercise

Use the CLI interface, pull an *alpine* or *ubuntu* image

- **docker run –it alpine|ubuntu**

Expected result with –i param an interactive sessions starts, with –t param (combined in a –it) a terminal starts

Open another terminal and run:

- **docker ps**

Expected result: the running container is shown

- **Run some commands in the terminal, e.g. ls, pwd, whoami**.

To stop the terminal (and the container) execute: **exit**

While the container is running, after issuing some commands in its terminal, run:

- **docker logs CONTAINER_NAME**

Expected result: outputs of commands issued in the container terminal

add –f to continue following the output of logs.

Other container commands

- **docker top CONTAINER_NAME**

list of all running processes inside the container

- **docker rm CONTAINER_NAME**

Removes container - can be done within Docker Desktop

Doubts ?

- docker --help
- docker COMMAND --help

Docker run

Common options when running containers:

```
# Run in detached (background) mode
docker run -d myapp:latest

# Map host port 8080 to container port 5000
docker run -p 8080:5000 myapp:latest

# Mount a volume
docker run -v $(pwd)/data:/app/data myapp:latest

# Set environment variables
docker run -e DEBUG=true myapp:latest

# Give the container a name
docker run --name app_container myapp:latest

# Remove container when it exits
docker run --rm myapp:latest
```


Exercise

Basic Docker Commands

- Pull the official Python image
- Run a container interactively
- Create a simple Python script inside the container
- Exit and restart the container
- Check - is the file still there?

Containers and communications

Container networking let containers to connect to and communicate with each other, or to non-Docker workloads.

Containers have networking enabled by default, and they can make outgoing connections.

A container has no information about what kind of network it's attached to, or whether their peers are also Docker workloads or not.

A container only sees a network interface with an IP address, a gateway, a routing table, DNS services, and other networking details.

To make a container accessible to other containers, we can enable inter-container communication by connecting the containers to the same network.

Docker Networks

Docker networks allow containers to communicate with each other:

Types of networks:

- **bridge**: default network for containers on the same host
- **host**: use host's networking directly
- **overlay**: for containers across multiple Docker hosts
- **macvlan**: assign MAC address to containers

Docker Networks

Common commands related to networks:

```
# Create a network
docker network create mynetwork

# Run container on network
docker run --network=mynetwork myapp:latest

# List networks
docker network ls

# Inspect network
docker network inspect mynetwork
```


Containers and ports

By default, when you run a container using docker run, the container doesn't expose any of its ports to the outside world.

- Use the **--publish** or **-p** flag to make a port available to services outside of Docker.

This creates a firewall rule in the host, mapping a container port to a port on the Docker host to the outside world. Publishing a container's ports makes them available not only to the Docker host, but to the outside world as well !

```
# Map port 8080 on the Docker host to
# TCP port 80 in the container.
-p 8080:80

# Map port 8080 on the Docker host
# IP 192.168.1.100 to TCP port 80 in the container.
-p 192.168.1.100:8080:80

# Only the Docker host (127.0.0.1) can access the
# published container port 80 through host port 8080
-p 127.0.0.1:8080:80
```

Exercise

- Get a nginx image
- Run it in daemon mode, mapping the port 80 of the image to port 8899 of the host
- Open localhost:<port> using the mapped port
- Optional: assign a name to the container

BUILD A DOCKER IMAGE

Building a Docker image

Docker images are built from a **Dockerfile** and a **"context"**.

A build's context is the set of files located in the specified PATH or URL. The build process can refer to any of the files in the context.

The URL parameter can refer to three kinds of resources: Git repositories, pre-packaged tarball contexts, and plain text files.

Use the docker build command to start building, specifying the path context:

docker image build [OPTIONS] PATH | URL | -

Use . i.e. the current working directory, as context to limit the build time.

Dockerfile basics

A Dockerfile is a text file containing instructions to build a Docker image:

```
# Use an official Python runtime as a parent image
FROM python:3.12-slim

# Set the working directory
WORKDIR /app

# Copy requirements and install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the current directory contents
COPY . .

# Make port 5000 available
EXPOSE 5000

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]
```

Dockerfile basics

It's a simple way to automate the image creation process.

The commands you write in a Dockerfile are almost identical to their equivalent Linux commands: this means you don't really have to learn new syntax to create your own dockerfiles.

Dockerfile directives

- **FROM**

The from directive is used to set base image for the subsequent instructions. A Dockerfile must have FROM directive with valid image name as the first instruction.

FROM ubuntu:20.04

- **WORKDIR**

The WORKDIR directive used to sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD commands during build.

WORKDIR /opt

Dockerfile directives

•**COPY**

The COPY directive used for copying files and directories from host system to the image during build. For example, in the following, the first commands will copy all the files from hosts html/ directory to /var/www/html image directory, while the second command will copy all files with extension .conf to /etc/apache2/sites-available/ directory.

COPY html/ /var/www/html/*

*COPY *.conf /etc/apache2/sites-available/*

•**RUN**

Using RUN directing ,you can run any command to image during build time. For example, you can install required packages during the build of image.

RUN apt-get update

RUN apt-get install -y apache2 automake build-essential curl

Dockerfile directives

•**ADD**

Similar to COPY, but it can also copy from URLs and automatically unpack compressed archives.

•**EXPOSE**

The EXPOSE directive indicates the ports on which a container will listen for the connections. After that you can bind host system port with container and use them. The EXPOSE instruction doesn't publish the port. It’s a type of documentation.

EXPOSE 80

EXPOSE 443

•#

Comments start with # (possibly continue with \). There are no block comments.

Dockerfile directives

- **CMD**

The CMD directive is used to run the service or software contained by your image, along with any arguments during the launching the container. There can only be one CMD instruction in a Dockerfile. CMD uses following basic syntax

CMD ["executable","param1","param2"]

For example, to start Apache service during launch of container, Use the following command.

CMD ["apachectl", "-D", "FOREGROUND"]

Dockerfile directives

- **ENTRYPOINT**

allows you to configure a container that will run as an executable. The preferred syntax is the exec form:

ENTRYPOINT ["executable", "param1", "param2"]

CMD can be used to pass additional arguments to ENTRYPOINT. Command line arguments to docker run <image> will be appended after all elements in an exec form ENTRYPOINT, and will override all elements specified using CMD.

ENTRYPOINT ["ps", "-au"]

CMD ["-x"]

Dockerfile directives

Example:

```
ENTRYPOINT ["python3", "script.py"]
CMD ["--help"]

docker run image            # Executes: python3 script.py --help
docker run image --verbose  # Executes: python3 script.py --verbose
```


Dockerfile directives

- **ENV**

The ENV directive is used to set environment variable for specific service of container. The ENV instruction allows for multiple <key>=<value> ... variables to be set at one time. The environment variables set using ENV will persist when a container is run from the resulting image. You can view the values using docker inspect, and change them using *docker run --env <key>=<value>*

ENV PATH=$PATH:/usr/local/pgsql/bin/

ENV PG_MAJOR=9.6.0

- **VOLUME**

The VOLUME directive creates a mount point with the specified name and marks it as holding externally mounted volumes from native host or other containers.

VOLUME ["/data"]

.dockerignore file

The *.dockerignore* file is used to exclude files or folders that we don't want to be included in the Docker build from being sent to the Docker daemon before the build.

- Similar in spirit to *.gitignore*
- This should go in the folder where the Dockerfile was placed.

Exercise

Exercise 1 - Sample docker file

- Write and run this “Hello World” Dockerfile:
  - FROM alpine
  - CMD ["echo", "Hello World!"]

Build and run it:

- docker build -t hello_world .
- docker run --rm hello_world

--rm deletes automatically the container once it has ended !

This will output:

Hello World!

Exercise

Exercise 1b – push image to Hub

- docker tag hello_world <docker_hub_username>/hello_world
- docker push <docker_hub_username>/hello_world

- Check the availability of the image on the Hub using Docker Desktop
- Delete the local image and then pull it from the hub
- docker pull <docker_hub_username>/hello_world

To delete the repository related to the image go to the Hub web page, select the repository and delete it in Settings tab or pick a specific tag in Tags tab and delete it.

Image layers

Each instruction in a Dockerfile results in a new image layer, which can also be used to start a container.

The new layer is created by starting a container using the image of the previous layer, executing the Dockerfile instruction and saving a new image.

Each instruction results in a static image — essentially just a filesystem and some metadata — and all the running processes in the instruction will be stopped.

This means that even long-lived processes, such as databases or SSH daemons in a RUN instruction, they will not be running when the next instruction is processed, or a container is started.

ENTRYPOINT or CMD instruction are the only way to start a service or process with the container.

Image layers

- Docker images are made up of multiple layers.
  - Each of these layers is a read-only filesystem.
  - A layer is created for each instruction in a Dockerfile and sits on top of the previous layers.

- When an image is turned into a container (from a docker run or docker create command), the Docker engine takes the image and adds a read-write filesystem on top.

- Because unnecessary layers bloat images, a good practice is to minimize the number of layers, e.g. by specifying several UNIX commands in a single RUN instruction.

Exercise

Exercise 3 + 3b – Flask apps

- Build the Docker images and run the resulting containers using the 3_container_python and 3_container_python_libs subprojects of the tutorial
- Check the apps using the links that will be shown running the Flask apps
- Test the port bindings and change them
- Test the use of ENV params and files

DATA PERSISTENCE

Data persistence

Docker containers provide you with a writable layer on top to make changes to your running container.

But these changes are bound to the container's lifecycle: If the container is deleted (not stopped), you lose your changes.

Let's take a hypothetical scenario where you are running a database in a container without any data persistence configured.

You create some tables and add some rows to them: but, if some reason, you need to delete this container, as soon as the container is deleted all your tables and their corresponding data get lost.

Docker provides us with a couple of solutions to persist your data even if the container is deleted. The two possible ways to persist your data are:

- Bind Mounts
- Volumes

Bind mounts

Bind mounts have been around since the early days of Docker.

When you use a bind mount, a file or directory on the host machine is mounted into a container.

The file or directory is referenced by its absolute path on the host machine.

By contrast, **when you use a volume**, a new directory is created within Docker's storage directory on the host machine, and Docker manages that directory's contents.

The file or directory does not need to exist on the Docker host already.

It is created on demand if it does not yet exist.

Bind mounts are very performant, but they rely on the host machine's filesystem having a specific directory structure available.

New Docker applications should use **named volumes** !

Docker volumes

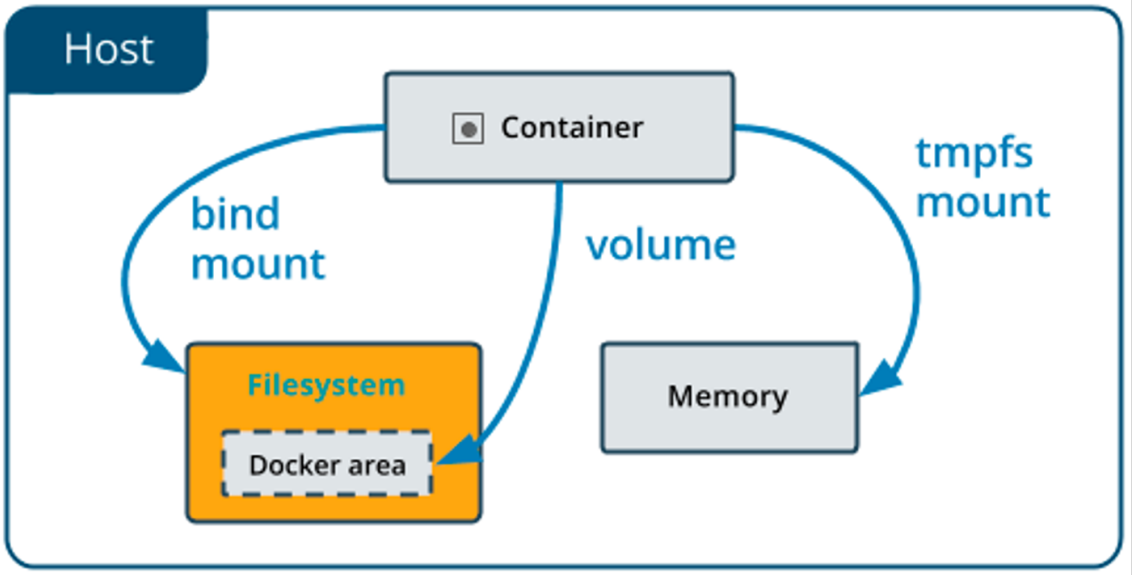
Volumes are the preferred mechanism for persisting data generated by and used by Docker containers.

While bind mounts are dependent on the directory structure and OS of the host machine, volumes are completely managed by Docker.

Volumes have several advantages over bind mounts:

- Volumes are easier to back up or migrate than bind mounts.
- You can manage volumes using Docker CLI commands or the Docker API.
- Volumes work on both Linux/macOS and Windows containers.
- Volumes can be more safely shared among multiple containers.
- Volume drivers let you store volumes on remote hosts or cloud providers, to encrypt the contents of volumes, or to add other functionality.
- New volumes can have their content pre-populated by a container.

Bind mounts vs volumes



Docker volumes

```
# Create a named volume
docker volume create my-volume

# Launch a container with volume
docker run -v /hostpath:/containerpath[:ro] …
# -v mounts the location /hostpath from the host filesystem
# at the location /containerpath inside the container
# With the ":ro" suffix, the mount is read-only
```

Purposes:

- store persistent data outside the container
- provide inputs: data, config files, . . . (read-only mode)
- inter-process communication (unix sockets, named pipes)

Exercise – named volume

Try to create a volume and launch a container with it

```
docker volume create my-volume

docker run --rm -t -i -v my-volume:/vol busybox
# In the terminal type:
# echo foo > /vol/bar
# CTRL+D

docker run --rm -t -i -v my-volume:/vol busybox cat /vol/bar
# Expected result: what was echoed inside

docker volume inspect my-volume
docker volume ls
docker volume rm my-volume
```


Exercise – bind mount

Bind a local temp directory to a busybox container, e.g.
docker run -v /tmp/test_bind_mnt:/tmp/test_bind_mnt -ti busybox

In the busybox terminal:
cd /tmp/test_bind_mnt
touch file.txt
echo foo > file.txt
exit

Check the host directory and cat the content of the file.txt

DOCKER COMPOSE

Docker compose

Docker Compose is a tool that was developed to help define and share multi-container applications.

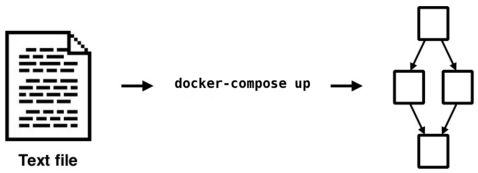
With Compose, we can create a YAML file to define the services and with a single command, can spin everything up or tear it all down.

Each of the containers is run in isolation but can interact with each other when required. Docker Compose defines a multi-container application in a single file.

docker-compose.yml

Structure of a docker-compose file

- Version # now deprecated, you'll get a warning !
- Services
  - Build
  - Image
  - Environment
  - Ports
- Volumes
- Networks



group.yml

```
name: counter

containers:
  web:
    build: .
    command: python app.py
    ports:
      - "5000:5000"
    volumes:
      - .:/code
    links:
      - redis
  redis:
    image: redis:latest
```


docker-compose.yml file example

```
version: "3.7"
services:
  app:
    image: node:12-alpine
    command: sh -c "yarn install && yarn run dev”
    ports:
      - 3000:3000
    working_dir: /app
    volumes:
      - ./:/app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos
  mysql:
    image: mysql:5.7
    volumes:
      - mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos
volumes:
  mysql-data:
```

Docker Compose commands

Basic Docker Compose commands:

```
# Start services defined in docker-compose.yml
docker-compose up

# Run in detached mode
docker-compose up -d

# Stop services
docker-compose down

# Stop services and remove volumes
docker-compose down -v

# View logs
docker-compose logs

# Execute command in a service container
docker-compose exec web python manage.py migrate
```


Docker Compose commands

docker-compose up –d

Create and start all the containers listed in the docker-compose.yml. –d starts them in daemon/detached mode. Each service is started according to the order and the specified dependencies. Without –d the containers are stopped when pressing CTRL+C

docker-compose stop

Stop the containers. Containers are NOT removed and can be restarted with docker-compose start.

docker-compose ps

List all the containers belong to the compose environment instance

docker-compose down

Stop and remove containers

Docker Compose commands

build	Build or rebuild services
help	Get help on a command
kill	Kill containers
logs	View output from containers
port	Print the public port for a port binding
ps	List containers
pull	Pulls service images
rm	Remove stopped containers
run	Run a one-off command
scale	Set number of containers for a service
start	Start services
stop	Stop services
restart	Restart services
up	Create and start containers

Exercise 1 – start a docker-compose.yml

Use the CLI to start a Docker Compose YAML file in the 3_container_python subproject

Create an .env file, add env variables

Try to build the image instead of pulling it in the compose file

Compose YML examples

•build | image

Pulling an image or building it

•If the image does not exist on the platform, Compose attempts to pull it

•Alternatively, we can build the image

```
services:
  jenkins:
    image: jenkins/jenkins:lts
    ports:
     - “8080:8080”
     - “50000:50000”
  myapp:
    build: ./dir
```


Compose YML examples

•Dependancy

depends_on

•Control the order of service startup and shutdown.

It is useful if services are closely coupled, and the startup sequence impacts the application's functionality.

```
services:
 web:
  build: .
  depends_on:
   - db
   - redis
 redis:
  image: redis
 db:
  image: postgres
```

Compose YML examples

•Volumes

Define mount host paths or named volumes that are accessible by service containers. You can use volumes to define multiple types of mounts; volume, bind, (and other – check docs).

•If the mount is a host path and is only used by a single service, it can be declared as part of the service definition. To reuse a volume across multiple services, a named volume must be declared in the volumes top-level element.

•The example shows a named volume (db-data) being used by the backend service, and a bind mount defined for a single service.

```
services:
 backend:
  image: example/backend
  volumes:
   - type: volume
     source: db-data
     target: /data
     volume:
      nocopy: true
      subpath: sub
   - type: bind
     source:
/var/run/postgres/postgres.sock
     target:
/var/run/postgres/postgres.sock

volumes:
 db-data:
```


Exercise – a more complex

- Use CLI to start the Docker compose in the 4_stack_lamp subproject
- See how volumes are used for data persistency and to sync the code between IDE and deployment/test

Docker Security

Important security considerations:

- Scan images for vulnerabilities (docker scan)
- Use minimal base images (alpine, slim)
- Don't run containers as root
- Limit container capabilities
- Use read-only filesystems where possible
- Sign images with Docker Content Trust
- Apply resource limits
- Use security scanning tools (Trivy, Clair)
- Keep Docker engine and images updated
- Implement proper network segmentation

Docker Scout - Vulnerability Scanning

Docker directly provides a vulnerability scanner named Docker Scout in Docker Desktop.

```
# Docker Scout (integrated with Docker Desktop)
docker scout quickview image_name:tag

# Scan with detailed vulnerabilities
docker scout cves image_name:tag

# Generate SBOMs (Software Bill of Materials)
docker scout sbom image_name:tag
```


User Privileges and Security Contexts

Containers can be run as non-root by creating a user inside the image and using it with the USER directive. In alternative, it can be specified at runtime (note that the image has to be prepared to run as non-root anyway).

```
# Create a user in the Dockerfile
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser

# Or specify user at runtime
docker run --user 1000:1000 image_name
```

Limit container capabilities

Docker usually enable all capabilities. It is a good practice to only enable needed ones:

```
# Drop all capabilities and add only needed ones
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE image_name

# Run with read-only filesystem
docker run --read-only image_name
```


Secrets management

Using Docker secrets (Swarm mode)

```
# Create a secret
docker secret create api_key api_key.txt

# Use secret in service
docker service create --secret api_key image_name
```

Using environment files

```
# Create .env file (don't commit to version control)
# Use with Docker Compose
docker-compose --env-file .env up
```

Using Docker Compose secrets (Swarm mode)

```
secrets:
  db_password:
    file: ./db_password.txt
  api_key:
    external: true
```

Container isolation

We can segment the network for each service:

```
# Docker Compose example
services:
  frontend:
    networks:
      - frontend
  backend:
    networks:
      - frontend
      - backend
  database:
    networks:
      - backend

networks:
  frontend:
  backend:
    internal: true  # Not exposed to outside world
```


Resource limits

Each container can have limits on the available resources:

```
# Memory and CPU constraints
docker run --memory=2g --cpus=2 image_name

# With Compose
services:
  ml-api:
    deploy:
      resources:
        limits:
          cpus: '2'
          memory: 2G
```

Minimize images

Use minimalistic base images, do not install unnecessary packages

- E.g. Alpine, -slim, etc.

The benefits are:

- Reduced size
- Less attack surface

- Use .dockerignore to exclude non relevant files from build

- Use multi-stage builds
  - There's a build phase that uses a large base image to compile stuff and a runtime phase that copies the resulting app in a "lighter" base image
  - See how volumes are used for data persistency and to sync the code between IDE and deployment/test

Deterministic images

- Ideally a Docker Image should be deterministic, i.e. should always produce the same result, but if we rely on latest versions…
- Specify precisely the versions of the dependencies

  Pin base image versions, use:

  FROM alpine:3.19

  instead of

  FROM alpine

- FROM alpine:3.19@ sha256:c5b1261d6d3e43071626931fc004f70149baeba2c8ec672bd4f27761f8e1ad6b

instead of

FROM alpine:3.19

- So that 3.19 tag refers to a specific version and not to 3.19.1, 3.19.2 etc. Get the sha256 from the repo.
- Specify the versions of the packages in requirements.txt

Secure images

- Do not run “root” processes
- Manage stop signals to shutdown gracefully
  - CMD ["app", "arg"] (exec form) instead of CMD app arg (shell form) results in more responsiveness since we avoid that the running shell might handle differently the signals. The exec form gets a PID 1 and receives signals directly.
  - Same applies to using docker exec –it CONTAINER my_command instead of docker exec –it CONTAINER sh –c “my_command”
- Use ENV variables to allow personalization of secret keys, instead of storing them in .env files included in the image
  - It would be like storing secret keys in a Github repo !

Ephemeral containers

- It should be possible to destroy containers and rebuild them anew from images
- No data should be stored in containers, plan the architecture of the app accordingly
  - E.g. store in S3 bucket, on MySQL servers, etc.
- Many SaaS like Vercel, Heroku, etc. treat the Dockers as such

Makefiles for repetitive commands

•Makefiles can be used to handle repetitive commands, e.g. assign targets to build, start-up containers or docker-compose tasks, etc.

•Pros:

- •Automation: make COMMAND helps to run a certain command without having to remember all the parameters
- •Documentation: reading the Makefile targets we understand what to do
- •Consistency: the same commands are used by everybody

•Cons:

- •The Makefile syntax

Multi-stage Dockerfile

Multi-stage builds help to create images:

- •Faster, parallelizing the build
- •Smaller, creating a final image with a smaller memory footprint from the artefacts created in the previous build stage

A build stage is represented by a FROM instruction, this is extended adding a new AS <stage-name> to it.

Use multiple FROM statements in the Dockerfile. Each FROM instruction can use a different base, and each of them begins a new stage of the build. Copy artifacts from one stage to another, leaving behind everything you don't want in the final image.

Multi-stage Dockerfile (reduce size)

```
# Stage 1: Build Environment
FROM builder-image AS build-stage
# Install build tools (e.g., Maven, Gradle)
# Copy source code
# Build commands (e.g., compile, package)

# Stage 2: Runtime environment
FROM runtime-image AS final-stage

# Copy application artifacts from the build stage (e.g., JAR file)
COPY --from=build-stage /path/in/build/stage /path/to/place/in/final/stage
# Define runtime configuration (e.g., CMD, ENTRYPOINT)
```


Multi-stage Dockerfile (parallelize)

```
FROM xxxx AS base
# install build tools
# copy source code

FROM base AS build-client
# build client part of the system

FROM base AS build-server
# build server part of the system, should be independent from client

FROM xxxx/base
COPY --from=build-client /bin/client /bin/
COPY --from=build-server /bin/server /bin/
```

Multi-stage dockerfile example

```
# Build stage
FROM node:16 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```


Exercise

Build both a single stage and a multiple stage Dockerfile in subproject 6_multistage.

Check the differences in terms of image size

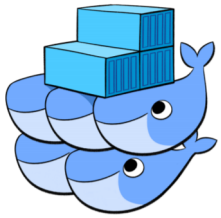
Use –f to specify the multistage Dockerfile that has a non-default name

Container Orchestration

Key features:
- Automated deployment
- Scaling
- Load balancing
- Self-healing
- Rolling updates

For managing containers at scale:

Docker Swarm:
- Built into Docker
- Simpler setup for small to medium deployments
- Integrated with Docker CLI



Kubernetes:
- Industry standard for container orchestration
- More features and flexibility
- Better for complex, large-scale deployments
- Steeper learning curve

Monitoring Containers

Tools and approaches for monitoring Docker containers:

Built-in commands:

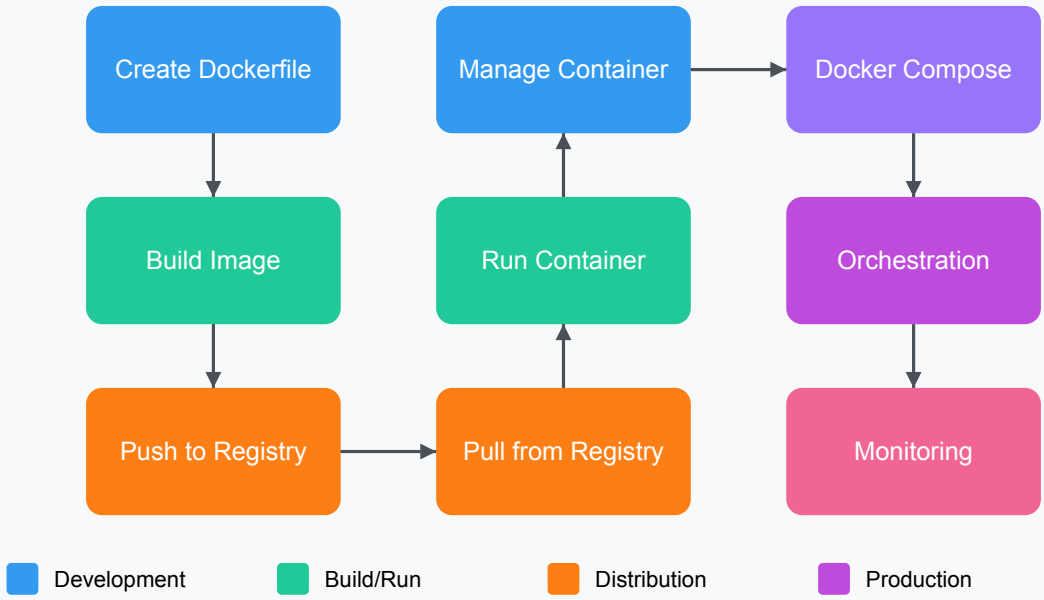
```
# Container resource usage
docker stats

# Container logs
docker logs container_id

# Container processes
docker top container_id
```

Monitoring Containers

External tools:
- Prometheus + Grafana
- cAdvisor
- Datadog
- Portainer

Metrics to monitor:
- CPU and memory usage
- Disk I/O
- Network traffic
- Container health
- Application-specific metrics

Docker Workflow for ML Applications



Dockerizing a Flask application

Example of Dockerizing a simple Flask application:

```
myapp/
├── app.py
├── requirements.txt
└── Dockerfile
```

```python
# app.py

from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5000)
```

```
# requirements.txt

flask==2.0.1
```


Dockerizing a Flask application

Example of Dockerizing a simple Flask application:

```
myapp/
├── app.py
├── requirements.txt
└── Dockerfile
```

```
# Dockerfile

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000
CMD ["python", "app.py"]
```

Docker for Development

Docker can also be used for development workflows.

Benefits:
- Consistent environments for all developers
- "Works on my machine" problems eliminated
- Easy onboarding of new team members
- Simplified testing with different dependencies
- Rapid development-to-production cycle
- Isolation from host system

Example workflow:
- Development in containers with mounted source code
- Hot reloading with mapped volumes
- Debugging inside containers
- Integration tests with Docker Compose

Practical Example: ML Application

A complete example for a machine learning application:

```
# Dockerfile

# Base image with Python and ML libraries
FROM python:3.12-slim

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Set up working directory
WORKDIR /app

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy model and application code
COPY models/ ./models/
COPY app/ ./app/

# Expose the API port
EXPOSE 8000

# Run the application
CMD ["python", "app/main.py"]
```

Practical Example: ML Application

```yaml
# docker-compose.yml

version: '3'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./app:/app/app
    environment:
      - MODEL_PATH=/app/models/model.pkl
      - LOG_LEVEL=INFO

  notebook:
    image: jupyter/scipy-notebook
    ports:
      - "8888:8888"
    volumes:
      - ./notebooks:/home/urix/work
      - ./data:/home/urix/data
```


Docker Desktop Dev Environments

Dev environments are shareable, reproducible development environments, based on Docker Compose to share exact environments between team members.

- Configurable via *devcontainer.json*
- Integrates with VS Code, JetBrains IDEs
- Under heavy development

Benefits:

- Share exact environments between team members
- Onboard new developers in minutes
- Consistent tooling across the team
- Version control your development environment

Docker Desktop Dev Environments

Can be specified in VSCode with a *devcontainer.json*

```
// devcontainer.json
{
  "name": "ML Development Environment",
  "dockerComposeFile": "docker-compose.yml",
  "service": "ml-workspace",
  "workspaceFolder": "/workspace",
  "extensions": [
    "ms-python.python",
    "ms-toolsai.jupyter",
    "ms-python.vscode-pylance"
  ],
  "settings": {
    "python.linting.enabled": true,
    "python.formatting.provider": "black"
  }
}
```


Additional resources

- Docker Documentation - https://docs.docker.com/
- Docker Hub - https://hub.docker.com/
- Docker Curriculum - https://docker-curriculum.com/
- Play with Docker - https://labs.play-with-docker.com/
- Docker Community Forums - https://forums.docker.com/
- Docker GitHub Repository - https://github.com/docker
- Docker Blog - https://www.docker.com/blog/

FLASK

What is Flask?

Flask is a lightweight web framework for Python that allows you to build web applications quickly and with minimal code.

Created by Armin Ronacher in 2010, Flask is considered a "micro" framework because it doesn't require particular tools or libraries, giving developers flexibility in their implementation choices.

Key features:

- Minimal core with extensible architecture
- Built-in development server and debugger
- RESTful request dispatching
- Support for secure cookies (client-side sessions)
- Template engine (Jinja2)
- Unit testing support

Setting Up Flask

Flask can be installed with pip as usual:

```
# Install Flask using pip
pip install Flask

# Simple Flask application
from flask import Flask

# Create a Flask app instance
app = Flask(__name__)

# Define a route
@app.route('/')
def hello_world():
    return 'Hello, World!'

# Run the application
if __name__ == '__main__':
    app.run(debug=True)
```

The __name__ parameter determines the root path of the application, which is used to find resources like templates and static files.

Routing in Flask

Routing in Flask maps URLs to specific functions that handle requests:

```
@app.route('/')
def home():
    return 'Welcome to the homepage!'

@app.route('/about')
def about():
    return 'About this website'

# Dynamic routing with parameters
@app.route('/user/<username>')
def show_user_profile(username):
    return f'User: {username}'

# Specifying the parameter type
@app.route('/post/<int:post_id>')
def show_post(post_id):
    return f'Post: {post_id}'
```

Flask supports various URL parameter types: string (default), int, float, path, uuid.

HTTP Methods

Flask routes can handle different HTTP methods:

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        # Handle form submission
        username = request.form.get('username')
        password = request.form.get('password')
        # Process login logic
        return f'Logged in as {username}'
    else:
        # Display login form
        return render_template('login.html')
```

Common HTTP methods:

- GET: Retrieve data (default)
- POST: Submit data
- PUT: Update existing data
- DELETE: Remove data

Templates

Flask uses **Jinja2** as its template engine:

app.py

```python
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/hello/<name>')
def hello(name):
    return render_template('hello.html', name=name)
```

hello.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Hello Page</title>
</head>
<body>
    <h1>Hello, {{ name }}!</h1>
    {% if name == 'admin' %}
        <p>Welcome, administrator!</p>
    {% else %}
        <p>Welcome, user!</p>
    {% endif %}
</body>
</html>
```

Jinja2 supports inheritance for templates. It allows to reuse common elements:

base.html

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}{% endblock %}</title>
    <link rel="stylesheet"
          href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <nav>
        <a href="{{ url_for('home') }}">Home</a>
        <a href="{{ url_for('about') }}">About</a>
    </nav>
    <div class="content">
        {% block content %}{% endblock %}
    </div>
    <footer>
        &copy; 2025 My Flask App
    </footer>
</body>
</html>
```

page.html

```
{% extends "base.html" %}

{% block title %}My Page{% endblock %}

{% block content %}
    <h1>Welcome to my page</h1>
    <p>This is a child template that extends the base template.</p>
{% endblock %}
```


Request and Response Objects

They contain form data, parameters from the URL and simplify response.

```
from flask import Flask, request, jsonify, redirect, url_for

app = Flask(__name__)

@app.route('/form', methods=['GET', 'POST'])
def form_handler():
    if request.method == 'POST':
        # Access form data
        name = request.form.get('name')
        email = request.form.get('email')

        # Access URL parameters
        page = request.args.get('page', 1)

        # Access JSON data (for API requests)
        if request.is_json:
            data = request.get_json()

        # Return different response types
        return jsonify({"name": name, "email": email})
        # Or redirect:
        # return redirect(url_for('thank_you'))

    return render_template('form.html')
```

Static Files

Flask serves static files (CSS, JavaScript, images) from a directory named 'static':

```
# In your templates, reference static files with url_for
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
<script src="{{ url_for('static', filename='js/script.js') }}"></script>
<img src="{{ url_for('static', filename='images/logo.png') }}">
```

A basic directory structure is:

```
my_flask_app/
├── app.py
├── static/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── script.js
│   └── images/
│       └── logo.png
└── templates/
    └── index.html
```


Error handling

Error are handled with another decorator:

```
@app.errorhandler(404)
def page_not_found(e):
    return render_template('404.html'), 404

@app.errorhandler(500)
def server_error(e):
    return render_template('500.html'), 500
```

Flask Sessions

Sessions allow you to store user-specific data between requests:

```python
from flask import Flask, session, redirect, url_for

app = Flask(__name__)
app.secret_key = 'your_secret_key'  # Required for sessions

@app.route('/')
def index():
    # Count the number of visits
    session['visits'] = session.get('visits', 0) + 1
    return f'You have visited this page {session["visits"]} times'

@app.route('/login')
def login():
    session['logged_in'] = True
    session['username'] = 'user123'
    return redirect(url_for('dashboard'))

@app.route('/logout')
def logout():
    # Remove specific session data
    session.pop('username', None)
    # Or clear all session data
    session.clear()
    return redirect(url_for('index'))
```


Structuring Flask Applications

For larger applications, it's recommended to use a package structure:

```
my_flask_app/
├── app/
│   ├── __init__.py       # Setup application, import blueprints
│   ├── models.py         # Database models
│   ├── routes.py         # Routes/views
│   ├── static/           # Static files
│   └── templates/        # HTML templates
├── config.py             # Configuration settings
└── run.py                # Application entry point
```

Example of RESTful APIs with Flask

Flask is ideal for creating RESTful APIs.

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# Sample data
books = [
    {'id': 1, 'title': 'Flask Web Development', 'author': 'Miguel Grinberg'},
    {'id': 2, 'title': 'Python Crash Course', 'author': 'Eric Matthes'}
]

# Get all books
@app.route('/api/books', methods=['GET'])
def get_books():
    return jsonify({'books': books})
```


Example of RESTful APIs with Flask

```python
# Get a specific book
@app.route('/api/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    book = next((book for book in books if book['id'] == book_id), None)
    if book:
        return jsonify({'book': book})
    return jsonify({'error': 'Book not found'}), 404

# Create a new book
@app.route('/api/books', methods=['POST'])
def create_book():
    if not request.is_json:
        return jsonify({'error': 'Invalid content type'}), 400

    data = request.get_json()

    if 'title' not in data or 'author' not in data:
        return jsonify({'error': 'Missing required fields'}), 400

    new_book = {
        'id': len(books) + 1,
        'title': data['title'],
        'author': data['author']
    }

    books.append(new_book)
    return jsonify({'book': new_book}), 201
```

Deploying Flask Applications

Flask applications can be deployed in various ways:

1. Using Web Server Gateway Interface (WSGI) servers

```
pip install gunicorn
gunicorn -w 4 -b 0.0.0.0:8000 "app:create_app()"
```


Deploying Flask Applications

Flask applications can be deployed in various ways:

2. Docker

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app:create_app()"]
```

Deploying Flask Applications

Flask applications can be deployed in various ways:

3. Cloud platforms:
    - Heroku
    - AWS Elastic Beanstalk
    - Google App Engine
    - Microsoft Azure App Service

4. Nginx + Gunicorn:
    - Nginx serves static files and acts as a reverse proxy
    - Gunicorn handles the Python application

Exercises

Exercise 1: Basic Flask Application

Create a simple Flask application that:

- Has a homepage with a welcome message
- Includes a route /about that displays information about the course
- Contains a route /time that displays the current date and time

Exercise 2: Dynamic Routes

Extend your application to include:

- A route /student/<name> that greets the student by name
- A route /multiply/<int:x>/<int:y> that displays the product of two numbers
- A route /upper/<word> that converts a string to uppercase

Final Exercises

Dockerize a Python Application

- Create a simple Flask application (reuse the one from the previous exercise)
- Write a Dockerfile
- Build and run the image
- Map a port to access the application from your host
- Modify the application and rebuild

Multi-container Application

- Create a docker-compose.yml file with:
    - A web application
    - A database service
- Define volumes for persistent data
- Define environment variables
- Start the services and verify they work together